

DYNAMIC PROGRAMMING

```
def knapsack(weights, values, capacity):
    n = len(weights)

    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):

            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

n = int(input("Enter number of items: "))

weights = list(map(int, input("Enter weights: ").split()))
values = list(map(int, input("Enter values: ").split()))

capacity = int(input("Enter capacity: "))

result = knapsack(weights, values, capacity)

print("Maximum value:", result)

print("Time Complexity: O(n × W)")
print("Space Complexity: O(n × W)")
```

```
Enter number of items: 5
Enter weights: 20
Enter values: 4
Enter capacity: 30
Maximum value: 4
Time Complexity: O(n × W)
Space Complexity: O(n × W)
```